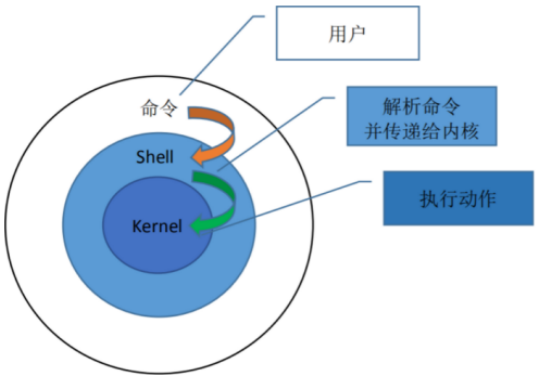
Shell脚本编程

一、Shell基础知识

1.Shell概述

Shell是一个命令解释器，它的作用是 解释执行用户输入的命令及程序 ，用户每输入一条命令，Shell就解释执行一条。这种从键盘一输入命令，就可以立即得到回应的对话方式，称为交互的方式。

Shell存在于操作系统的最外层，负责与用户直接对话，把用户的输入解释给操作系统，并处理各种各样的操作系统的输出结果，然后输出到屏幕返回给用户。输入系统用户名和密码并登录到Linux后的所有操作都是由Shell解释与执行的。



理解了Shell之后，在理解Shell脚本就简单了。当命令或程序语句不在命令行下执行，而是通过一个程序文件来执行时，该程序就被称为Shell脚本。如果在Shell脚本里内置了很多条命令、语句及循环控制，然后将这些命令一次性执行完毕，这种通过文件执行脚本的方式称为非交互的方式。Shell脚本类似于DOS系统下的批处理程序（早期扩展名为.bat），用户可以在Shell脚本中敲入一系列命令及命令语句组合。这些命令、变量和流程控制语句等有机的结合起来，就形成了强大的Shell脚本。

Shell应用场景：

① 简化传统运维流程

② 编写日常巡检脚本

③ 软件安装配置流程化

④ 方便进行日志分析与处理 => 软件日志 => 系统日志/var/log

案例：入门案例

```
#!/bin/bash
cd /var/log
cat /dev/null > messages
echo "Log cleaned up."
```

代码解析：

```
#!/bin/bash代表声明Shell解析器（代表将来下方代码默认会调用/bin/bash进行解析）
cd切换目录
/dev/null空设备，读取里面的内容，
> 重定向到某个文件，代表对文件进行清空操作 /var/log/message
echo输出
```

由于是Shell脚本程序，通常期后缀名.sh，代码编写完成后，还需要为其添加x权限，通过./脚本执行

小结：

① Shell解析器：解析用户输入的命令或者程序

② Shell应用场景：简化运维流程、日常巡检、批量化软件安装与配置等等

2. Shell编程中的变量

① 变量是存储数据的容器

② 变量中存储的数据是临时的

③ 变量在程序运行过程中是可以发生变化的

基本语法：

```
变量=变量的值
```

注意：=日常生活中叫做等号，但是我们脚本中，将其称之为赋值运算符；执行顺序从右向左执行

示例代码：

```
[root@itheima ~] # name='itheima'
[root@itheima ~] # echo $name
Shift + 4 = $
```

> 注意事项：=等号两边不能有空格！ 变量名可以自定义，但是要满足一定的命名规则。

总结：

变量核心作用？存储数据

变量有三句话：① 变量存储数据的容器 ② 变量中存储的数据是临时的 ③ 变量中存储的数据可以发生变化

3. 变量的命名规则

变量名称只能由 `字母、数字或下划线` 组成

不能以 `数字` 开头

严格 `区分大小写`

不能使用Shell中保留的关键字 => test、if、for

判断题：

① `$1` 是一个合法的Shell变量名，对么？（错误）

② `my_var` 是一个合法的Shell变量名，对么？（正确）

③ `2nd_var` 是一个合法的Shell变量名，对么？（错误）

④ `MY-VAR` 是一个合法的Shell变量名，对么？（错误）

总结：

变量的命名规则，变量不能随意命名，只能是字母 + 数字 + 下划线组成，不能以数字开头，区分大小写

建议使用英文单词或者拼音作为变量名称

4. 变量的种类

变量可以分为两大类：环境变量（全局变量）和 普通变量（局部变量）

环境变量也可称为全局变量，可以在创建它们的Shell及其派生出来的任意子进程Shell中使用，环境变量又可以分为自定义环境变量和bash内置的环境变量。

普通变量也可以称为局部变量，只能在创建它们的Shell函数或Shell脚本中使用。普通变量一般由开发者在开发脚本程序时创建。

```
export env_name=tom
echo $env_name  -- tom
bash -- 开启子任务
echo $env_name --tom
---------------------------------
age=100
echo $age   --100
bash  -- 开启子任务
echo $age   --查询无结果(普通变量子任务不能使用)
```

5. 环境变量

环境变量是操作系统用来 存储系统和用户配置信息的变量 ，Shell 和应用程序通过它来获取重要信息，例如当前用户、命令路径、终端类型等。Shell 通过这些变量确保命令能正确执行。

Java => 运行环境 => JDK

Java程序运行过程中，可以通过环境变量自动寻找JDK相关信息，方便程序运行

环境变量的特点：

① 系统全局性：环境变量对所有进程有效，任何程序（如编辑器、Shell 脚本等）都可以使用这些变量。

② 临时与永久：在命令行中定义的环境变量只在当前会话中有效，一旦退出，变量就会消失。如果希望永久保存，需要将变量添加到特定的配置文件中。=> /etc/profile

按照系统规范，所有环境变量的名字均采用 大写 形式。在将环境变量应用于用户进程程序之前，都应该用export命令导出定义。

例如：正确的环境变量定义方法为 `export MYHOME=/home/itheima`

有一些环境变量，比如 `HOME` （用户家目录）、 `PATH` （命令搜索路径）、 `SHELL` （Shell类型）、 `USER` （用户名）、 `PWD` （当前工作目录）等，这些变量在用户登录时由系统自动设置。

如果想要设置环境变量，就要在给变量赋值之后或在设置变量时使用export命令，如下图所示：

```
export  变量名=变量的值
变量名=变量的值  ;  export 变量名
```

除了export命令，我们还可以使用declare –x完成同样的功能。

```
declare  -x  变量名=变量的值
```

> 注意：以上设置请不要在变量名前面添加$符号

☆ 临时设置：可以在命令行中直接设置，但退出会话后会丢失。

```
export MYHOME=/home/itheima
```

☆ 永久设置：如果希望变量在每次登录时都有效，需要将其添加到配置文件中：

```
用户级别：~/.bash_profile 或 ~/.bashrc
```

```
系统级别：/etc/profile 或 /etc/bashrc
```

所有环境变量名称通常采用大写字母，且必须使用 `export` 命令导出，才能让其对子进程有效。

总结：

在Shell脚本编程中，变量一共分为（环境变量）和（普通变量/局部变量）？

环境变量在定义时，建议变量名称（全大写）

配置环境变量时，如果在命名行通过export定义，是临时的还是永久的？临时的

如果想配置为永久，可以放置在哪些文件？（`/etc/profile`、`/etc/bashrc`、`~/.bash_profile`或`~/.bashrc`）

6. 案例：生产环境下JDK环境变量的配置

回顾rpm命令：离线安装、卸载

搜索软件

```
rpm  -qa | grep java
-q : query查询
-a : all所有
```

安装软件

```
rpm -ivh xxx.rpm
-i : install安装
-vh: 显示安装进度
```

卸载软件

```
rpm -e  java-1.8.0-openjdk-headless-1.8.0.362.b09-4.el9.x86_64
-e: 普通卸载
--nodeps: 强制卸载（有风险）
```

传统配置

```
# 清理系统自带JDK环境
rpm -e java-1.8.0-openjdk-headless-1.8.0.362.b09-4.el9.x86_64 --nodeps

```

```
# 上传压缩包到Linux
# 创建工作目录并解压jdk
mkdir -p /export/server
tar -xf jdk-8u241-linux-x64.tar.gz -C /export/server/

# vim /etc/profile
export JAVA_HOME=/export/server/jdk1.8.0_241
export PATH=$PATH:$JAVA_HOME/bin
export CLASSPATH=.:$JAVA_HOME/lib/dt.jar:$JAVA_HOME/lib/tools.jar

# 让环境变量生效
source /etc/profile
```

设置JDK环境变量

```
# vim jdk.sh
#!/bin/bash
#卸载系统自带的JDK
rpm -e java-1.8.0-openjdk-headless-1.8.0.362.b09-4.el9.x86_64 --nodeps
#创建目录
mkdir -p /export/server
#对JDK软件包解压
tar -xf jdk-8u241-linux-x64.tar.gz -C /export/server/
#配置JDK环境变量 => 把配置信息 => /etc/profile
echo 'export JAVA_HOME=/export/server/jdk1.8.0_241' >> /etc/profile
echo 'export PATH=$PATH:$JAVA_HOME/bin' >> /etc/profile
echo 'export CLASSPATH=.:$JAVA_HOME/lib/dt.jar:$JAVA_HOME/lib/tools.jar'
>> /etc/profile
#刷新/etc/profile文件
source /etc/profile
```

让以上配置立即生效

```
source jdk.sh
```

验证JDK是否配置成功

```
java -version
```

> 注意：如果Shell脚本中包含source命令，则脚本的执行必须采用source 脚本名称.sh

7. 显示与取消环境变量

```
echo $HOME          # 用户的家目录
echo $UID            # 当前用户的UID，相当于id -u
echo $PWD            # 当前工作目录的绝对路径名
echo $SHELL         # 当前SHELL
echo $USER           # 当前用户
```

```
printf "$HOME\n"    # printf是一个格式化打印输出工具
```

用env（printenv）显示默认环境变量：

```
env
```

用set也可以显示环境变量（包括局部变量）：

```
set -a 变量名 -- 把变量输出到环境变量
```

显示环境变量

```
echo $USER
itheima
```

用unset消除本地变量和环境变量

```
unset USER
echo $USER
```

> unset移除变量时，不需要添加$美元符号，直接使用变量名称即可

总结：

获取变量信息 => （echo $变量名称）

移除/消除某个变量 => （unset 变量名称）

8. 环境变量初始化与对应文件的生效顺序

/etc/profile、/etc/bashrc、~/.bash_profile、~/.bashrc

在登录Linux系统并启动一个bash shell时，默认情况下bash会在若干个文件中查找环境变量的设置。这些文件可统称为系统环境文件。bash检查的环境变量文件的情况取决于系统运行Shell的方式。系统运行Shell的方式一般有3种：

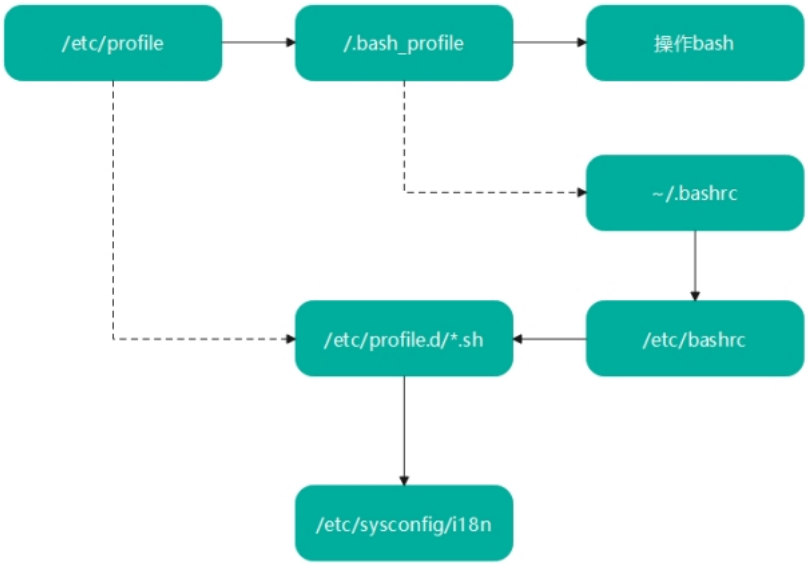
① 通过系统用户登录后默认运行的Shell（/bin/bash）

② 非登录交互式运行Shell => cron定时调度调用脚本

③ 执行脚本运行非交互式Shell

当用户登录Linux系统时，Shell会作为登录Shell启动。

此时登录Shell加载环境变量顺序：



用户登录系统后首先会加载/etc/profile全局环境变量文件，这是Linux系统上默认的Shell主环境变量文件。系统上每个用户登录都会加载这个文件。

当加载完/etc/profile文件后，才会执行/etc/profile.d 目录下的脚本文件，这个目录下的脚本文件有很多，例如：系统的字符集设置(/etc/sysconfig/i18n) 等。

之后开始运行$HOME/.bash_profile$(用户环境变量文件)，在这个文件中，又会去找HOME/.bashrc

(用户环境变量文件)，如果有，则执行，如果没有，则不执行。在$HOME/.bashrc 文件中又会去 找/etc/bashrc (全局环境变量文件)，如果有，则执行，如果没有，则不执行。

如果用户的Shell不是登录时启动的(比如手动敲下bash时启动或者其他不需要输入密码的登录及远程SSH连接情况)，那么这种非登录Shell只会加载$HOME/.bashrc$(用户环境变量文件)，并会去找$/etc/bashrc$(全局环境变量文件)。因此如果希望在非登录$Shell$下也可读到HOME/.bashrc 或者/etc/bashrc, 而不是$HOME/.bash_profile 或/etc/profile。

小结：

> 用户环境变量配置在：$HOME/.bashrc文件中

> 全局环境变量配置在：/etc/bashrc文件中

> 不管用户还是全局的环境变量，建议不要放在$HOME/.bash_profile和/etc/profile中

中午留一个小作业：把MySQL安装封装成一个Shell脚本

要求1：能自动安装MySQL软件

要求2：把随机密码放入/root/mysql_pass.txt文件中

9. 案例： Shell设置登陆提示的两种方式

第一种方式是在/etc/motd里增加提示的字符串，如下：

```
[root@itheima ~] # cat /etc/motd   # 文件里仅为字符串内容
welcome to itheima yunwei training.
```

第二种方式是在/etc/profile.d/下面增加如下脚本：

```
[root@itheima ~] # cat /etc/profile.d/myshell.sh  # myshell.sh为此目录下的脚本文件
echo 'here is itheima yunwei training'
```

> 注意事项：若要在登陆后初始化或显示加载内容，则把脚本文件放在/etc/profile.d/下即可（无须加执行权限）

10. 单引号与双引号的区别

在定义文本（字符串）时，单引号和双引号效果一致的，但是如果定义的文本内容中存在变量时。两者有所不同：

① 单引号中的变量无法解析，系统会原样输出（把变量当做一个文本信息）

② 双引号中的变量是可以正常解析的，系统会把变量先翻译为对应的内容，然后进行显示

变量定义时，变量的内容可以用单引号或双引号引起来，也可以不加引号。

```
a=192.168.88.100
b='192.168.88.100'
c="192.168.88.100"

echo "a=$a"
echo "b=$b"
echo "c=${c}"
```

结论：针对字符串类型的数据，是否添加引号，其效果完全一样！

```
a=192.168.88.200
b='192.168.88.200'
c="192.168.88.200"

a=192.168.88.100-$a
b='192.168.88.100-$b'
c="192.168.88.100-$c"

echo "a=$a"
echo 'b=$b'
echo "c=${c}"
```

结论：

① 第一种形式，变量$a可以正常解析

② 第二种形式，单引号中的变量无法正常解析，而是原样输出

③ 第三种形式，双引号中的变量可以正常解析

> 注：数字内容的变量定义可以不加引号，其他没有特别要求的字符串等定义最好都加上双引号，如果真的需要原样输出就加单引号，定义变量加双引号是最常见的使用场景。

11. 把命令执行结果赋值给变量

把一个命令的结果作为变量的内容赋值的方法

对需要获取命令结果的变量内容赋值的常见方法有两种：

```
变量名=`ls`   # ``反撇号
变量名=$(ls)   # $(命令)
```

案例演示：

```
CMD1=`ls`
echo $CMD
```

```
CMD2=$(pwd)
echo $CMD2
```

```
CMD3=$(date +%F)
echo $CMD3

%F: 年-月-日
%Y-%m-%d: 年-月-日
```

小结：

可以把命名的执行结果赋值给变量 =>

变量=`命令`

变量=$(命令)

12. Shell中特殊且重要的内置变量

在Shell中存在一些特殊且重要的变量，例如0、1、$#，我们称之为特殊位置参数变量。要从命令行、函数或脚本执行等处传递参数时，就需要在Shell脚本中使用位置参数变量。

变量	含义
$0	当前脚本的文件名。
$n （n≥1）	传递给脚本或函数的参数。n 是一个数字，表示第几个参数。例如，第一个参数是 1，第二个参数是2。
$#	传递给脚本或函数的参数个数。
$*	传递给脚本或函数的所有参数，参数是一个整体，每一个变量参数之间以空格隔开。
$@	传递给脚本或函数的所有参数。参数是独立的，也是全部输出
$?	上个命令的退出状态或函数的返回值
$$	当前 Shell 进程 ID。对于 Shell 脚本，就是这些脚本所在的进程 ID。

案例演示：

```
[root@itheima ~]# cat 1.sh
#!/bin/bash
echo "$0 = $0"
echo "$# = $#"
echo "$* = $*"
echo "$@ = $@"
echo "$1 = $1"
echo "$2 = $2"
echo "$3 = $3"
echo "$11 = ${11}"
echo "$12 = ${12}"
```

> 注意：如果想在Shell中输出一些特殊的符号，如$、#、等等，建议使用反斜杠进行转义，就是把这些符号当做一个普通在字符直接输出（不进行系统层面的解析）

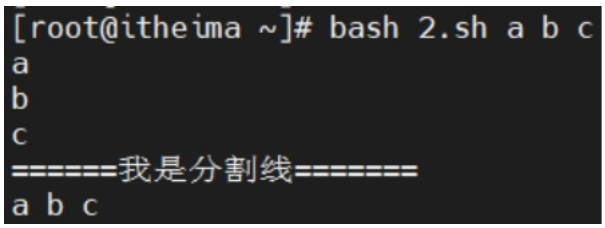
案例演示：*与@区别

```
[root@itheima ~] cat 2.sh
#!/bin/bash
for i in "$@"
do
echo $i
done

echo "======我是分割线======="

for i in "$*"
do
echo $i
done
```

运行结果：



总结：`$*和$@有何区别`？

相同点：都可以用于接收所有的参数

不同点：

$*是把所有参数当做一个整体，参数之间空格隔开

$@虽然也可以接收所有参数，但是其每个参数保存时依然是独立的

13. Shell运算符

算术运算：默认情况下，Shell就只能支持简单的整数运算

+ - * / %（取模，求余数）

Bash Shell 的算术运算有四种使用方式:

```
① 使用 $(( 表达式 )) => 新版，推荐表达式两边保留一个空格
② 使用$[ 表达式 ]      => 老版，推荐表达式两边保留一个空格
③ 使用 expr 外部程式，乘法、括号需要加反斜杠转义 => expr 1 \* 2 => 注意：运算符两边必须有空格
④ 使用let 命令，赋值并运算，支持++、--，++在原有变量基础上+1，--在原有变量值基础上减-1
```

案例：

```
echo $(( 1+1 ))
echo $(( 2-1 ))
echo $(( 2*3 ))
echo $(( 4/2 ))
echo $(( 9%4 ))
```

++ 与 -- 运算符：++就是在变量本身+1，--就是在变量本身-1

```
i=1
let i++
echo $i
```

```
j=1
let ++j
echo $j
```

++ 或 -- 对表达式的影响：

```
unset i j
i=1;j=1
let x=i++      先赋值，再运算 => echo $x 1
let y=++j      先运算，再赋值 => echo $y 2
```

Shell运算符是否能实现小数运算呢？答：默认不可以，但是可以结合bc命令实现。

```
echo 1+1.5|bc
2.5
```

> 注：CentOS Stream 9中没有bc命令，可以通过`dnf install bc -y`

总结：

整数运算 => (())或[]，如果有++或--的情况，建议使用let

默认不支持小数运算 => `dnf install bc -y` => `echo 1+1.5 |bc`

二、Shell编程小工具（重点）

1、cut工具

作用：cut是列截取工具，用于列的截取

语法：

```
# cut 选项  文件名
```

常见选项：

```
-c: 以字符为单位进行分割,截取
-d: 自定义分隔符，默认为制表符\t
-f: 与-d一起使用，指定截取哪个区域
```

举例说明:

```
# cut -d: -f1 1.txt              以:冒号分割，截取第1列内容
# cut -d: -f1,6,7 1.txt      以:冒号分割，截取第1,6,7列内容
# cut -c4 1.txt                    截取文件中每行第4个字符
# cut -c1-4 1.txt                截取文件中每行的1-4个字符
# cut -c4-10 1.txt             截取文件中每行的4-10个字符
# cut -c5- 1.txt                   从第5个字符开始截取后面所有字符
```

案例：提取出当前服务器上所有用户的用户名进行权限管理审查，但只需提取 `/etc/passwd` 文件中的用户名信息，而不需要其他字段。

解决方案：通过 `cut` 命令，使用 `:` 作为分隔符，仅提取用户名列。

```
# cut -d: -f1 /etc/passwd
```

小结:

cut作用？用于截取指定列的信息

-c：字符

-d、-f：-d指定分隔符，-f指定截取第几列

2、sort工具

作用：sort工具用于排序;它将文件的每一行作为一个单位，从首字符向后，依次按ASCII码值进行比较，最后将他们按升序输出。

语法和选项：`man sort`

```
-u : uniq去除重复行
-r : reverse降序排列，默认是升序
-o :  将排序结果输出到文件中,类似重定向符号>
-n : 以数字排序，默认是按字符排序
-------------------------------------------------
-t : 分隔符
-k : 第N列
-R : 随机排序，每次运行的结果均不同
```

举例说明:

```
# sort -n -t: -k3 1.txt                按照用户的uid进行升序排列
# sort -nr -t: -k3 1.txt               按照用户的uid进行降序排列
# sort -n 2.txt                        按照数字排序
# sort -nu 2.txt                       按照数字排序并去重
# sort -nr 2.txt                       按照数字进行降序排列
# sort -nru 2.txt                      按照数字进行降序排列并去重
# sort -n 2.txt -o 3.txt               按照数字排序并将结果重定向到文件
# sort -R 2.txt                        随机排序
# sort -u 2.txt                        去除重复行
```

sort作用？就是用于日常排序操作，默认按照字符升序，可以使用（-n）参数按照数字排序？如果想降序，使用（-r）

3、uniq工具

> uniq用于去除连续的重复行

```
常见选项：
-i:  忽略大小写
-c: 在每一行的前面显示该行在文件中出现的次数 => 统计这一行出现频率频次 => 数据统计
-d: 只显示重复行

举例说明：
# uniq 2.txt
# uniq -d 2.txt
# uniq -dc 2.txt
```

案例：在 CentOS Stream 9 的服务器上，运维团队检测到多次失败的登录尝试，怀疑存在潜在的恶意入侵。为了提高服务器的安全性，运维团队需要快速找到频繁失败登录的IP地址，以便采取进一步的安全措施（如阻止IP、加强认证机制等）

第一步：从日志中提取失败登录的IP地址

使用 `grep` 查找失败的登录尝试： 登录失败的相关日志通常包含 "Failed password" 关键字。

```
grep "Failed password" /var/log/secure | cut -d ' ' -f 13
```

这里的 `cut -d ' ' -f 13` 表示使用空格作为分隔符，提取日志中第11个字段（假设IP地址在第13列）。这个命令将输出所有失败登录的IP地址。

第二步：统计重复的IP地址

使用 `uniq -c` 来统计每个IP地址的出现次数。

```
grep "Failed password" /var/log/secure | cut -d ' ' -f 13 | sort | uniq -
c
```

第三步：按登录失败次数排序

为了查看哪些IP地址尝试的次数最多，可以使用 `sort -nr` 来按次数进行降序排序。

```
grep "Failed password" /var/log/secure | cut -d ' ' -f 13 | sort | uniq -c | sort -nr
```

效果：

```
15  192.168.1.100
 8    203.0.113.55
 4    192.168.1.105
```

4、tee工具

tee工具是从标准输入读取并写入到标准输出（即屏幕）和文件，即：双向覆盖重定向（屏幕输出|文本输入）

应用场景：

实时监控并记录：可以在执行系统命令时，同时将结果显示在屏幕上和记录到文件中，用于后续分析。

保存日志文件：在执行长时间运行的命令时，可以实时查看输出并保存为日志，方便追踪问题。

结合管道使用：在复杂的管道命令中，将中间结果保存到文件中，同时继续传递给下一个命令。

```
选项：
-a 双向追加重定向
```

案例1：查看并保存系统状态

```
top -n 1 | tee system_status.log

注意：运行top 命令一次，输出系统当前的状态后退出，不会进行持续刷新
```

案例2：将输出追加到文件中

```
echo "hello world" | tee -a log.txt
```

总结：

双向操作：① 打印输出到屏幕 ② 把执行结果覆盖/追加到指定文件中

5、diff工具

`diff` 是 Linux 中用于比较两个文件或目录内容的工具。它逐行比较文件内容，并输出文件之间的差异，帮助用户识别两个文件在何处不同。`diff` 主要用于文件比较和合并，可以生成补丁文件，以便在需要时将一个文件的修改应用到另一个文件上。

基本语法

```
diff [选项] 文件1 文件2
```

默认情况下，`diff` 的输出格式为 **正常格式**，其符号含义如下：

- `<` 表示**文件1**中的内容
- `>` 表示**文件2**中的内容
- **c** 表示修改（change）
- **a** 表示增加（add）
- **d** 表示删除（delete）

案例1：比较file1.txt 和 file2.txt文件区别

```
diff file1.txt file2.txt
2c2
备注说明：
2左边文件行号
c（change）修改
2右边文件的行号
```

案例2：比较dir1 和 dir2目录区别

`diff` 也可以用于比较两个目录中的文件差异，默认会递归地比较子目录。

```
diff -r dir1 dir2
选项：-r选项代表递归操作
```

案例3：生成补丁文件（保证两个文件内容高度一致）

通过 `diff` 的统一格式，可以生成补丁文件，之后可以通过 `patch` 命令应用补丁。

```
diff -u original_file.txt modified_file.txt > changes.patch
```

生成的 `changes.patch` 文件包含了所有差异。可以通过 `patch` 命令将补丁应用到原始文件：

```
patch original_file.txt < changes.patch
选项：-u选项以统一模式显示差异，这种格式在创建补丁时常用
```

小结：

diff工具主要用于（文件）和（目录） 差异比较?

我们还可以借助于diff补丁功能，保证两个文件内容的高度一致!

6、paste工具

作用：`paste` 是 Linux 中用于**逐行合并文件**的命令行工具。它能够将多个文件的内容按行拼接，输出到标准输出或另一个文件中。与 `cat` 不同，`paste` 是将每个文件中的行拼接在一起，而不是逐文件合并。

一般用于多日志合并分析

```
常用选项：
-d：delimiter，自定义间隔符，默认是tab
-s：串行处理，非并行
```

准备数据

```
vim  file1.txt
a
b
c

vim  file2.txt
1
2
3
```

案例1：逐行合并

```
paste  file1.txt file2.txt
```

效果

```
a   1
b   2
c   3
```

案例2：使用自定义分隔符

使用 `-d` 选项可以指定分隔符，而不是默认的 `tab`。例如，用逗号作为分隔符：

```
paste -d ',' file1.txt file2.txt
```

效果

```
a,1
b,2
c,3
```

案例3：使用 `-s` 选项可以将文件内容串行拼接，即将文件的所有行拼接为一行

```
paste -s file1.txt
```

效果

```
a   b   c
```

运维场景：在服务器上进行日志分析时，运维人员需要将不同日志文件的内容按行合并，以便比较同一时间段内发生的事件。

解决方案：使用 `paste` 工具将多个日志文件按行拼接在一起，方便对比分析。

```
paste /var/log/messages /var/log/secure
2025-01-03 10:03:00   ERROR + Failed passwod
```

小结：

paste主要作用就是把文件进行（逐行）合并

实际生产环境中，通过把一些日志、文件进行逐行合并，方便分析。

7、tr工具

tr用于字符转换，替换和删除；主要用于 删除文件中控制字符 或进行 字符转换

主要用于字符删除与替换操作 => cpu 95% => 95，去掉%百分号

基本语法

```
用法1：命令的执行结果交给tr处理，其中set1用于查询，set2用于转换处理
# commands | tr  'set1'  'set2'
注：'set1'要替换或处理的字符集，'set2'将字符替换为的目标字符集

用法2：tr处理的内容来自文件，记住要使用"<"标准输入
# tr  'set1'  'set2' < filename

用法3：匹配string1进行相应操作，如删除操作
# tr options 'set1' < filename
常用选项：
-s (--squeeze-repeats)：将相邻的重复字符压缩成一个字符。‘hello
 world' => 'hello world'
-d (--delete)：删除输入中指定的字符。
```

常用正则表达式：

字符串	含义
a-z	匹配所有小写字母
A-Z	匹配所有大写字母
0-9	匹配所有数字
0-9a-zA-Z	匹配所有数字与大小写字母

案例1：小写转大写

```
echo "hello world" | tr 'a-z' 'A-Z'
```

案例2：删除文本中的数字字符

-d参数代表删除

```
echo "my number is 12345" | tr -d '0-9'
```

案例3：压缩重复字符

```
echo "too    many    spaces" | tr -s ' '
```

案例4：字符删除和转换结合使用

```
echo "linux 1234 commands" | tr -d '0-9' | tr 'a-z' 'A-Z'
```

tr运维场景：运维人员在处理日志文件时，发现日志中存在大量无关的数字字符（如时间戳中的数字），这些字符使日志难以分析。运维希望通过删除这些数字字符来简化日志内容。

解决思路：使用 `tr` 工具删除日志文件中的数字字符

```
cat /var/log/messages | tr -d '0-9'
```

小结：

tr工具主要可以进行（替换）和（删除）操作

三、Shell中的判断语句

判断语句：解决业务逻辑中的判断问题

1. 基本语法

```
语法格式一：
test  条件表达式

语法格式二：
[ 条件表达式 ]，注意：判断语句两边必须保留一个空格

语法格式三：
[[ 条件表达式 ]]  支持正则表达式，注意：判断语句两边必须保留一个空格
```

2. 文件存在与否判断

```
-e  exist，是否存在  不管是文件还是目录，只要存在，条件就成立
-f  是否为普通文件
-d  是否为目录
-S  socket，是否为套接字文件 => MySQL => mysql.sock这个文件就是套接字
-c  character，是否为字符设备
-b  block，是否为块设备文件，如/dev/sda
-L  link，是否为软链接文件

记住：-e、-f、-d、-L
```

> character是指直接与硬件设备进行交互的文件，按字符流的方式读写数据。字符设备通常用于访问串行端口、键盘、终端等设备，如/dev/tty终端设备，/dev/null空设备

三种语法格式：

```
test -e file            只要文件存在条件为真
[ -d /shell01/dir1 ]        判断目录是否存在，存在条件为真
[ ! -d /shell01/dir1 ]      判断目录是否存在,不存在条件为真
[[ -f /shell01/1.sh ]]      判断文件是否存在，并且是一个普通的文件
```

-s 判断文件是否有内容（大小），非空文件条件满足

说明：-s表示非空，！ -s 表示空文件

说明：1.sh文件里有内容的

```
[root@itheima]# test -s 1.sh
[root@itheima]# echo $?
0
[root@itheima]# touch aaa
[root@itheima]# cat aaa
[root@itheima]# test -s aaa
[root@itheima]# echo $?
1
[root@itheima]# test ! -s aaa
[root@itheima]# echo $?
0
[root@itheima]# test ! -s 1.sh
[root@itheima]# echo $?
1
```

3. 文件权限判断

itheima/readme.txt

```
-r  当前用户对其是否可读
-w  当前用户对其是否可写
-x  当前用户对其是否可执行
```

案例：

判断是否有读权限

```
test -r /etc/passwd
```

判断是否有写权限

```
test -w /tmp/testfile
```

判断是否有执行权限

```
test -x /bin/bash
```

4. 两个文件比较（新旧）

```
file1 -nt  file2    比较file1是否比file2新
file1 -ot  file2    比较file1是否比file2旧
file1 -ef  file2    比较是否为同一个文件，或者用于判断硬连接，是否指向同一个inode

-nt: new than
-ot: old than
-ef: equal file
```

案例演示：

```
test file1 -nt file2
[ file1 -ot file2 ]
```

小结：

通过新旧比较，了解两个文件的创建信息

5. 大小比较符号（重点）

在 Shell 中，`-eq`、`-ne`、`-gt`、`-lt`、`-ge` 和 `-le` 用于数值比较。它们常用于测试两个数字是否相等、不等、大小关系等。下面是这些操作符的详细解释和一些实际案例。

```
-eq (equal)相等
-ne (not equal)不等
-gt  (greater than)大于
-lt   (little than)小于
-ge (greater equal)大于等于
-le  (little equal)小于等于
```

案例：

```
a=5
b=5
test $a -eq $b
```

案例：

```
a=8
b=7
test $a -ge $b
```

小结：

由于在Shell脚本中，很多数学符号都有特殊含义，如果要进行日常比较，只能使用字母形式的比较符号！

6. 字符串之间的判断

```
-z                                          (zero)是否为空字符串          字符串长度为0，就成立
-n                                          (not zero)是否为非空字符串    只要字符串非空，就是成立

string1 = string2              是否相等
string1 != string2             不等
```

案例演示：

```
[root@itheima ~]# AAA=hello
[root@itheima ~]# BBB=world
[root@itheima ~]# test -z $AAA
[root@itheima ~]# echo $?
1

[root@itheima ~]# test -n $AAA
[root@itheima ~]# echo $?
0

[root@itheima ~]# [ $AAA = $BBB ]
[root@itheima ~]# echo $?
1

[root@itheima ~]# [ $AAA != $BBB ]
[root@itheima ~]# echo $?
0
```

小结：

-z、-n、=、!=，主要用于字符串判断操作

7. 逻辑判断符号

逻辑判断符号：

[表达式1 -a 表达式2]，两个条件同时满足，整个大条件为真

[表达式1] && [表达式2]，两个条件同时满足，整个大条件为真

故事：女孩子，自身条件比较好，要求男方必须要有车 且 有房，牵手成功

[有车 -a 有房]

[表达式1 -o 表达式2]，两个条件满足任意一个，整个大条件为真

[表达式1] || [表达式2]，两个条件满足任意一个，整个大条件为真

故事：女孩子，自身条件一般，要求男方必须要有车 或 有房，牵手成功

[有车 -o 有房]

```
-a  和 &&  (and 逻辑与)        两个条件同时满足，整个大条件为真
-o  和  ||     (or 逻辑或)           两个条件满足任意一个，整个大条件为真
```

生产应用：代替if...else语句

```
条件表达式 && 如果条件成立，执行此内容 || 如果条件不成立，执行此内容
执行原理：
第一步：把条件表达式 && 如果条件成立，执行此内容当做一个整体，先判断
如 [ 1 -eq 2 ] && echo true，如果左边是真的，必须要判断右边（执行右边），因为中间&&，必须两边同时为真，同时执行；如果左边是假的，则右边完全不需要执行了，因为逻辑与，必须两边同时成立，只要有一边为假，最终结果肯定是假的。

第二步：把第一步得到的结果 与 后面的条件组合
true || echo false，由于左边是真的，中间逻辑符号是||，则右边肯定不会执行
false || echo false，由于左边是假的，中间逻辑符号是||，要想获取最终结果必须执行右边表达式，才能获取最终结果
```

案例演示：

```
[ 1 -eq 1 -a 1 -ne 0 ]              整个表达式为真
[ 1 -eq 1 ] && [ 1 -ne 0 ]

[ 1 -eq 1 -o 1 -ne 1 ]              整个表达式为真
[ 1 -eq 1 ] || [ 1 -ne 1 ]

[root@itheima ~]# [ 1 -eq 0 ] && echo true || echo false
false

[root@ itheima ~]# [ 1 -eq 1 ] && echo true || echo false
true

&&:前面的表达式为真
||: 前面的表达式为假
```

小结：

-a 或 &&：逻辑与，两个条件同时满足，整个大条件为真

-o 或 || ： 逻辑或，两个条件满足任意一个，整个大条件为真

条件表达式 && 条件为真执行此语句 || 条件为假则执行此语句

四、场景案例：Linux运维日常巡检

1、任务场景

随着企业信息化和云计算技术的快速发展，Linux 系统已成为核心服务器平台。特别是 **CentOS Stream 9**，作为一个社区驱动的企业级操作系统，其稳定性和安全性要求较高。系统巡检和健康检查已经成为系统管理的重要组成部分。

在日常运维中，管理员往往需要定期执行一系列检查操作，例如：

- 检查 CPU、内存、磁盘的使用情况。
- 检查系统负载、活动进程和日志信息。
- 确保防火墙、SELinux 等安全设置的有效性。

目前，手动执行这些检查任务耗时且容易出错，因此需要一个自动化脚本来定期执行这些任务，并将检查结果保存为日志，以供运

2、任务拆解

2.1 日志文件路径定义

2.2 开始检查写入日志

2.3 系统负载检查 => uptime

2.4 CPU 使用情况检查 => top(需要自己处理，获取cpu使用率)

2.5 内存使用情况检查 => free -h

2.6 磁盘使用情况检查 => df -h

2.7 活动进程检查 => ps aux

2.8 最近的系统日志检查 => journalctl -n 10

2.9 系统更新检查 => dnf check-update

2.10 SELinux 状态检查 => sestatus

2.11 防火墙状态检查 => firewall-cmd --state

2.12 当前挂载的磁盘检查 => lsblk

2.13 Cron 任务列表检查 => crontab -l

2.14 系统基本信息检查 => hostname && uname -r

3、任务实施

```
#!/bin/bash

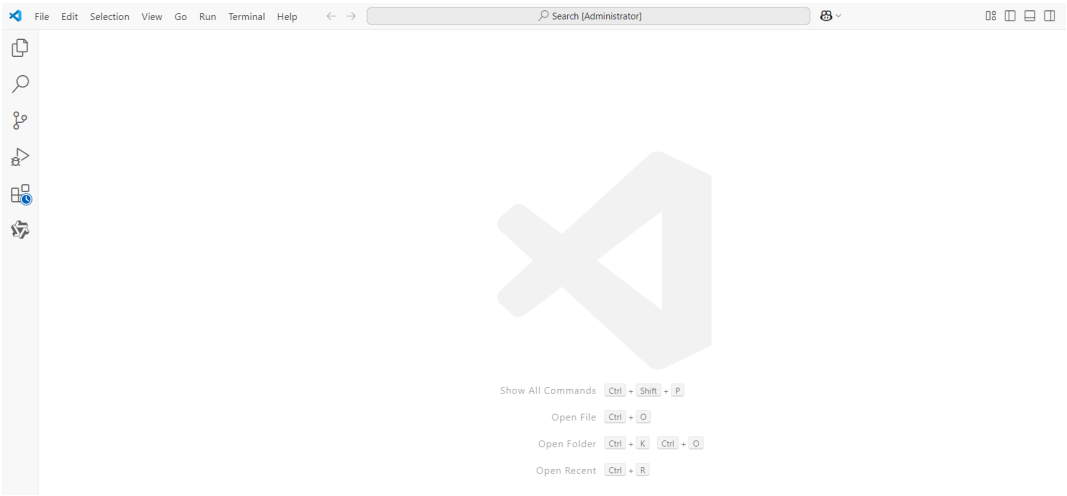
# 定义日志文件路径
LOG_FILE="/var/log/centos_stream_9_health_check.log"
current_time=$(date '+%Y-%m-%d %H:%M:%S')
```

```
6
7  # 写入日志函数
8  echo "$current_time - Starting daily health check..." >> $LOG_FILE
9
10 # 系统负载检查
11 echo "$current_time - System Load: $(uptime)" >> $LOG_FILE
12
13 # CPU 使用情况检查
14 echo "$current_time - CPU Usage: $(top -bn1 | grep 'Cpu(s)' | cut -d',' -f4 | cut -d' ' -f2 | awk '{print 100 - $1}')" >> $LOG_FILE
15
16 # 内存使用情况检查
17 echo "$current_time - Memory Usage: $(free -h)" >> $LOG_FILE
18
19 # 磁盘使用情况检查
20 echo "$current_time - Disk Usage: $(df -h)" >> $LOG_FILE
21
22 # 活动进程检查
23 echo "$current_time - Top 10 CPU consuming processes: $(ps aux --sort=-%cpu | head -n 10)" >> $LOG_FILE
24
25 # 网络连接状态检查
26 echo "$current_time - Network Connections: $(netstat -tuln)" >> $LOG_FILE
27
28 # 最近的系统日志检查
29 echo "$current_time - Recent System Logs: $(journalctl -n 10)" >> $LOG_FILE
30
31 # 系统更新检查
32 echo "$current_time - System Updates: $(dnf check-update)" >> $LOG_FILE
33
34 # SELinux 状态检查
35 echo "$current_time - SELinux Status: $(sestatus)" >> $LOG_FILE
36
37 # 防火墙状态检查
38 echo "$current_time - Firewall Status: $(firewall-cmd --state)" >> $LOG_FILE
39
40 # 当前挂载的磁盘检查
41 echo "$current_time - Mounted Disks: $(lsblk)" >> $LOG_FILE
42
43 # Cron 任务列表检查
44 echo "$current_time - Cron Jobs: $(crontab -l)" >> $LOG_FILE
45
46 # 系统基本信息检查
47 echo "$current_time - System Info: $(hostname && uname -a)" >> $LOG_FILE
48
49 # 写入日志结束
50 echo "$current_time - Health check completed." >> $LOG_FILE
51
52 exit 0
```

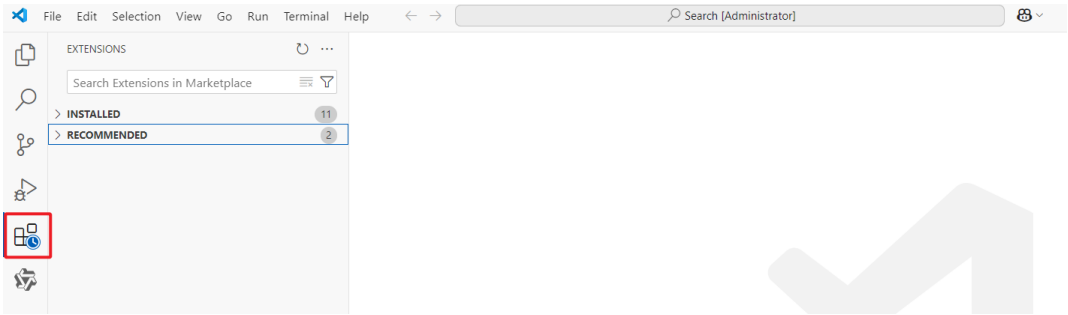
附录：AI大模型辅助运维开发

1、vscode + 通义大模型工具使用

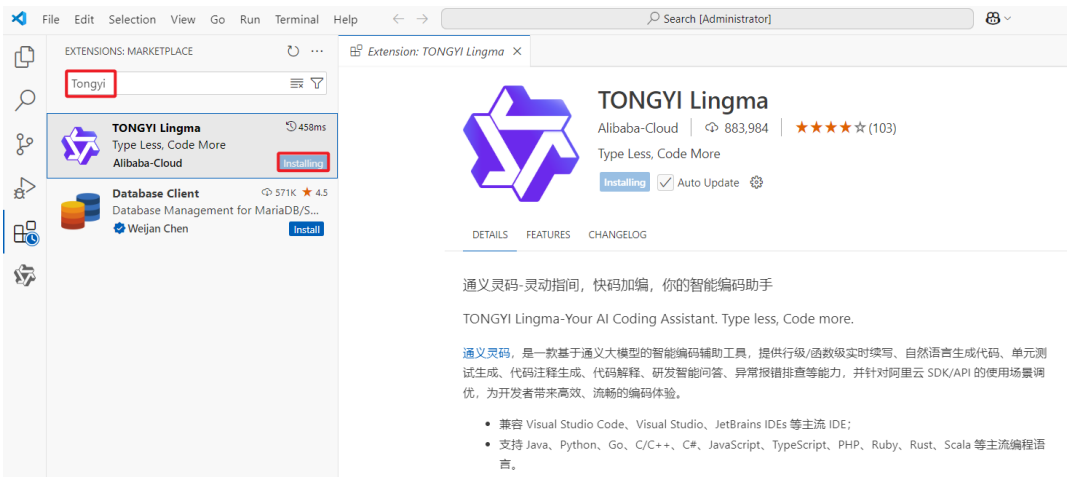
安装后打开，如下图所示：



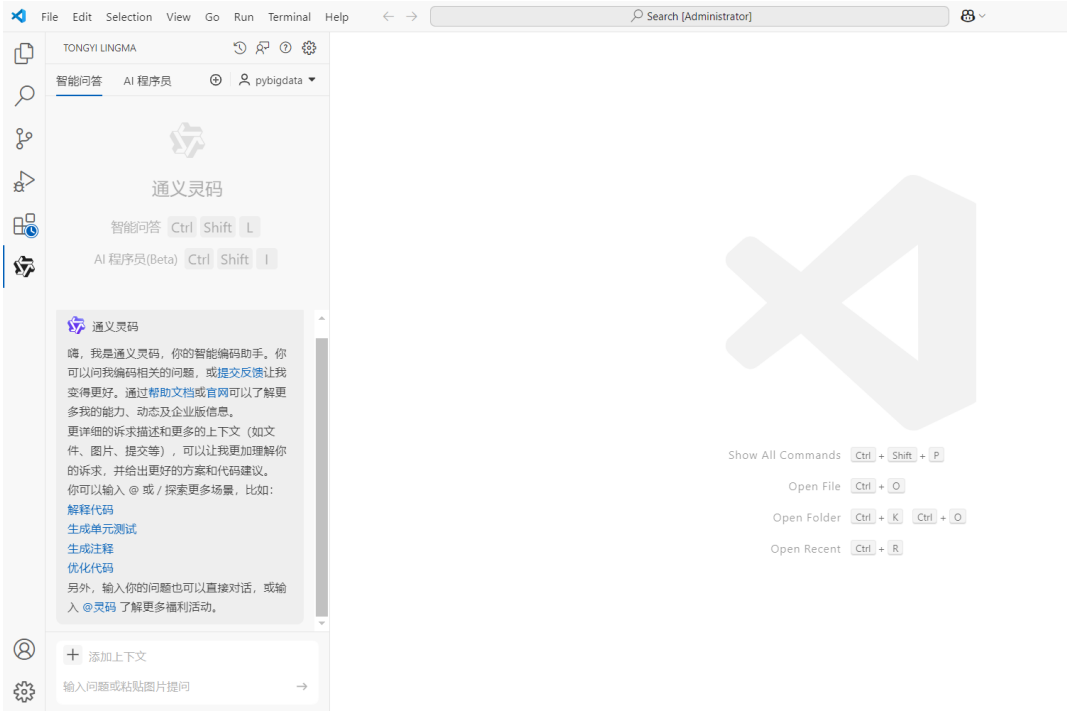
点击左下角扩展模块（积木小图标）



在搜索框中，输入Tongyi关键词，安装通义灵码插件



安装完成后，点击Login登录，使用阿里云账号即可登录并免费使用通义灵码工具



使用通义灵码，辅助运维开发工 作

shell04_ping.sh shell03_all.sh

shell03_all.sh

```
3  for i in "$*"
4  do
5    echo $i
6  done
7  #$@
8  echo '========================'
9  for j in "$@"
10 do
11   echo $j
12 done
```

注意：不要依靠大模型帮我们写代码，而是帮助我们提供一些程序设计思路，小提示

2、 kimi大模型实现课程总结

官网地址： https://kimi.moonshot.cn/

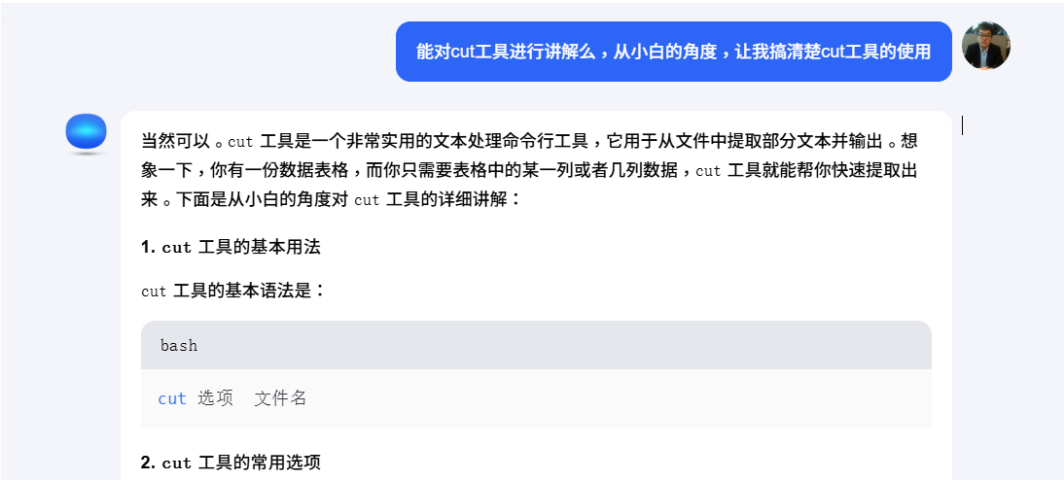


基于以上按钮上传课程讲义，然后让大模型帮助我们做核心内容的总结：





辅助学习：



3、 kimi大模型完成课程扩展

根据文档提供的知识点，出一些关于cut 、sort 、uniq等Shell测试题目



Shell脚本编程
MD, 35.9 KB

以下是一些关于 `cut`、`sort`、`uniq` 等 Shell 命令的测试题目，适合用来检验对这些工具的理解和应用能力：

1. `cut` 命令题目

题目1： 你有一个文本文件 `data.txt`，内容如下：

```
name:age:job
Alice:30:Engineer
Bob:25:Designer
Charlie:35:Manager
```